Project Name	Allixir Intelligence
Online team meeting	https://tu-berlin.zoom-x.de/j/68161739941?pwd=Tz3duoAQeQHGWAsBNMexRqNxlnSPCI.1
Production system (if any)	...
Test system (if any)	...
GitHub repository	https://github.com/amosproj/amos2026ss03-ailixir-intelligence
GitHub feature board	https://github.com/orgs/amosproj/projects/99
GitHub imp-squared backlog	https://github.com/orgs/amosproj/projects/103/views/1
Position of T-shirt logo	Upper Torso Right Small
Link to logo for black T-shirt	https://www.shirtinator.com/s/w1QJzfDfTC-gGBjVmnCscg
Link to logo for white T-shirt	https://www.shirtinator.com/s/sMNil0MoQzmuNuc04Y6Ljw
Additional materials	...
Team maling list	oss-amos-proj3@lists.fau.de

Last Name	First Name	GitHub User Name	Email Address
Brüggemann	Jonas	jonasbrue	jonas.brueggemann@campus.tu-berlin.de
Ebied	Andrew	andrewluka	a.ebied@campus.tu-berlin.de
Vinzens	Hannes	hannes-v	hannes.vinzens@fau.de
Ahmed	Hasnat	hasnatahmed331	hasnat.ahmed@fau.de
Busse	Tobias	aseuc	tobias.t.busse@fau.de
Sandt	Eloi	eloinoel	eloi.sandt@campus.tu-berlin.de
Amer	Hossam	Hossam-Amer	hossam.amer@fau.de
Zeeshan	Muhamad	MuhammadZeeshan2	sharif.zeeshan@fau.de
Haseeb	Abdul	AbdulHaseeb-SE	abdul.ah.haseeb@fau.de
Vogt	Marcel	VogtIndustries	marcel.vogt@campus.tu-berlin.de

		Product Owner					
#	Meeting Day	Review	Planning	Software Developer	Release Manager	Scrum Master	Comment
1	2026-04-15		Eloi	Everyone else	N/A	COACH student	
2	2026-04-22	Eloi	Tobias	Everyone else	Andrew Ebied	COACH student	
3	2026-04-29	Tobias	Eloi	Everyone else	Andrew Ebied	COACH student	
4	2026-05-06	Eloi	Tobias	Everyone else	Hasnat Ahmed	COACH student	
5	2026-05-13	Tobias	Eloi	Everyone else	Hossam Amer	COACH student	
6	2026-05-20	Eloi	Tobias	...	Muhamad Zeeshan	COACH student	
7	2026-05-27	Tobias	Eloi		Hannes Vinzens	COACH student	
8	2026-06-03	Eloi	Tobias		Marcel Vogt	COACH student	
9	2026-06-10	Tobias	Eloi		Abdul Haseeb	COACH student	
10	2026-06-17	Eloi	Tobias		Hannes Vinzens	COACH student	
11	2026-06-24	Tobias	Eloi		Hasnat Ahmed	COACH student	
12	2026-07-01	Eloi	Tobias		Hossam Amer	COACH student	
13	2026-07-08	Tobias	Eloi		Muhamad Zeeshan	COACH student	
14	2026-07-15	Eloi	/		Abdul Haseeb	COACH student	Demo day!
15	2026-07-22		/			COACH student	Retrospective
Product owners, software developers, and Scurm Master are set and ideally don't change over time; the critical part is the Release Manager role you need to define here							

Goals	Deliver MVP
	mutual and personal improvement
	deliver a maintainable prototype that can be potentially used as a starting point for the industry partner
Meeting norms	Meeting Should stick to deliverables.
	maybe once or twice per week. for each meeting, a document should be made that includes the points which we will go over. the document could be edited by anyone in case anyone has points they want to discuss in the meeting, although this might be particularly useful for POs.
	write an email in case you will be late/unable to attend team meetings
	be on time
	mute yourself in calls if you don't talk
	Protocol industry partner meetings
	Team Meeting starts at 12 (30 minutes earlier)
Working norms	
	deciding on a release manager should be part of the team meetings
	Would be cool if we could give an estimate of when we will be available to work, and if possible, also specify on which issues and till when we expect to be finished with them. in case of delays, i think they should be communicated as well so that the team gets a rough idea of when they could be done or if help is needed from other people
	SDs can self-assign issues at will; if that doesn't work out, issues will be assigned during team meetings
	Coding (style) guidelines, common AI agent setup, CI/CD linting/formating checks
	Documentation (how to run + code comments)
	Testing the written code/feature. All tests should pass before merging to main/master
	git flow strategy for working on code
	Definition of Done (DoD) A checklist of criteria (e.g. tests passed, code reviewed, documentation updated) that must be met before an issue is considered finished
	Definition of Ready (DoR); whether or not devs have enough capacity to take on an issue and finish it in time
	Pull Requests need to be reviewed and approved by at least one other developer
Coordination norms	
	Bigger decisions will be put to vote (ideally on slack)
	Work should be distributed with clear boundaries.
	use feature board actively and communicate with other people to coordinate work
Communication norms	
	replies within ~24h
	would also be cool if people communicate if they are unable to reply within 24h, with an estimate of when they will reply
	Communication with the industry partner over Slack

	it would also be nice to have dedicated communication channels for the team internally and with the industry partner. another dedicated channel for SDs as well
	agree on one main channel of communication: Slack
	Respect, active listening, tolerance
Consideration norms	
	Respect, Communicate Conflicts
	be kind and be cooperative
Cont. improvement norms	constructive criticism
	open feedback about things that could be improved
Rewards	
	eat ice cream together at the end of the course
Sanctions	
	gets brought up in the meeting
Signatures	
Scrum Master	Jonas Brüggemann
Product owner	Eloi Sandt
Product owner	Tobias Busse
Software developer	Hasnat Ahmed
Software developer	Hossam Amer
Software developer	Hannes Vinzens
Software developer	Muhammad Zeeshan
Software developer	Abdul Haseeb
Software developer	Andrew Ebied
Software developer	Marcel Vogt

Product Vision	Project Mission
Allixir Intelligence transforms unstructured documents into connected, actionable knowledge. Users can upload or scan documents (e.g., medical reports, invoices), from which the system extracts domain-specific, meaningful data fields and organises them into a personalised knowledge base. An AI-powered voice/chat interface enables users to ask precise, context-aware questions and better understand their personal documents, as well as relationships and developments across their document history. At its core, the platform is designed to be domain-agnostic and configurable for a wide range of use cases.	The mission of this project is to create an MVP that runs on mobile devices. Core functionality includes extracting structured information from uploaded documents, storing and organising it within a personal knowledge base, and enabling conversational querying using RAG with optional enrichment from external knowledge sources such as research papers. The system will demonstrate domain configurability using medical documents as the primary domain and finance as a secondary domain.

Term	Definition
AiLixir	A mobile application that lets users capture or upload documents, extract structured knowledge from them, and ask an AI questions about the analysed content.
User	A person who uses AiLixir to manage documents, start knowledge extraction, view results, and interact with the AI.
Account	A user identity that stores private app data and enables access to personalised documents, chats, and extracted knowledge.
Registration / Sign-up	The process by which a new user creates an account, typically using email, password, and required personal details.
Login	The process by which an existing user proves their identity and gains access to private app screens and backend functionality.
Authenticated User	A user who has successfully logged in and is allowed to access private documents, protected screens, and authenticated backend APIs.
Document	A user-provided file or scan that can be stored, previewed, analysed, and used as the source for knowledge extraction.
Captured Document	A document created by taking a picture or scan with the smartphone camera and passing the result back to the app.
Uploaded Document	A document selected from the device file system or submitted from the camera that becomes available inside AiLixir.
Document Preview	A small visual representation of a selected document or document page before the user starts analysis or extraction.
Document View	The screen that lists the documents stored for the authenticated user and acts as the entry point to document actions.
Document Details View	The screen for one document that displays metadata, pages, status, and actions such as starting extraction or downloading results.
Document Metadata	Descriptive information about a document, such as status, size, filename, document date, upload date, facility, and tags.
Document Page	An individual page or image belonging to a document, especially relevant for multi-page uploads or camera scans.
Document Tag / Label	A user-facing classification attached to a document, for example a lab result label, used to organize and filter documents.
Document Status	The current processing state of a document, such as not extracted, extraction in progress, or analysed.
Knowledge Extraction	The user-triggered process that converts document content into structured domain information, such as entities and relationships.
Extraction Status	An indicator showing whether knowledge has already been extracted from a document, is still being extracted, or has not been extracted yet.
Medical Document	A document containing medical information, such as a lab report, discharge summary, diagnosis information, medication information, or blood marker data.
Blood Biomarker	A measurable health value extracted from medical documents and displayed over time in the medical dashboard.
Medical Dashboard	A visual overview that shows extracted medical data, including biomarkers, historical trends, and filterable health information.
Financial Metric	A financial value extracted from documents or connected sources, such as balances, asset values, transaction data, or portfolio performance data.
Finance Dashboard	A visual overview that consolidates financial metrics and transactions and presents portfolio performance over time.
Structured Data	A machine-readable representation of extracted information, typically JSON or graph data, that downstream services can process reliably.
Domain-specific Entity	A meaningful object identified in a document, such as Patient, Diagnosis, Medication, MedicalReport, or a financial transaction.
Relationship	A typed connection between two domain-specific entities, such as HAS_DIAGNOSIS, PRESCRIBED_FOR, MENTIONS_DIAGNOSIS, or TREATED_BY.
Triple	A structured fact consisting of a subject entity, a relationship, and an object entity, prepared for knowledge graph ingestion.
Knowledge Graph	A structured network of entities and relationships extracted from documents and used to support retrieval, validation, visualisation, and LLM answers.
Node	An entity stored in the knowledge graph, such as a patient, diagnosis, medication, or document.
Edge	A relationship stored in the knowledge graph that connects two nodes and expresses how they are related.
Graph Database	Persistent storage for knowledge graph nodes and edges, implemented either as a dedicated graph database or another schema that can store triples.
Graph Ingestion	The act of writing extracted nodes and edges into the knowledge graph database so that they can be retrieved later.
Subgraph	A relevant subset of the knowledge graph returned for a specific query or prompt, usually consisting of targeted nodes and their direct edges.
Graph Retrieval	The process of finding and returning a query-relevant subgraph so that another service, especially the LLM integration, can use it as context.
LLM	A language model that generates answers for users and can use retrieved graph data as grounding context.
AI Agent	The conversational assistant in the app that receives user prompts and returns answers about uploaded and analysed documents.

Term	Definition
Prompt	A user input to the AI agent, typed or spoken, that asks for information or analysis about uploaded and analysed documents.
LLM Context	Structured information provided to the LLM before answer generation so that answers can reference verified extracted facts.
JSON Context Window	A JSON-formatted representation of graph facts that is injected into the LLM prompt for initial integration and validation.
Hallucination	An AI answer that is not supported by verified extracted facts; the knowledge graph is intended to reduce this risk.
GraphRAG Framework	A framework-based approach for extracting, storing, and retrieving graph-based knowledge for use with generative AI.
Manual JSON Pipeline	An extraction approach in which entities and relationships are first produced as structured JSON before being sent to an ingestion service.
Embedding	A vector representation of data sources such as PDFs, images, JSON files, or knowledge graphs, used for retrieval in a RAG setup.
RAG Database	A retrieval database that stores embeddings and supports finding relevant information for AI-generated answers.
Knowledge Graph Visualisation	A visual representation of a document's extracted graph, typically as a node-link diagram or structured report.
Knowledge Graph PDF Export	A generated PDF representation of a document's knowledge graph that the user can download and inspect locally.
Chat Screen	The app screen that displays user prompts and AI-agent responses and provides input controls for text and voice interaction.
Voice Dictation	A microphone-based input mode that turns spoken language into a text prompt for the AI agent.
Voice Conversation	A voice-based interaction mode in which the user can converse with the AI agent without relying only on typed prompts.
Backend API	A service interface that allows the frontend to authenticate users, store and retrieve documents, trigger extraction, and request generated results.
Document API	Backend functionality that accepts document uploads, returns the user's stored documents, and retrieves a document by document ID.
Persistent App Storage	Long-term storage for user accounts, chats, document identifiers, document metadata, and document files.
OCR	The process of converting text from scanned or photographed documents into machine-readable text before downstream extraction.

Sprint #	Sprint goal
1	None
2	None
3	None
4	None
5	Build the knowledge graph extraction pipeline for documents, provide API endpoints for knowledge extraction, implement the knowledge extraction UI in the app, and connect existing document views to real backend data through API integration.
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	

Sprint #	Story Points Realized
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	
	PLEASE CREATE THE VELOCITY CHART ON A NEW TAB USING THE DATA FROM THIS TAB

Sprint	Goal	Feature Name	Est. size	Est. remaining	Real size	Real remaining
Release						
Total			42	42		
Sprints						
1	Establish the project foundation by completing initial research, defining the app direction, and setting up the frontend development environment.		0	42	0	42
2	Deliver the first functional application prototype with document upload, authentication UI, architecture planning, and initial document analysis capabilities.		24	42	23	42
3	Build the core document management UI, define the app navigation structure, and implement document capture using the camera while establishing the backend cloud infrastructure.		18	18	18	19
4			0	0	0	1
5			0	0	0	1
6	WIP		0	0	0	1
Features						
1	**Establish the project foundation by completing initial research, defining the app direction, and setting up the frontend development environment.**	Setup documentation README.md				
		Research which extraction tool do we use? (Document AI, OCR, OCR with Google AI)				
		App Design Mockups / Wireframes / Moodboard				
		Frontend Setup Development Environment for App				
2	**Deliver the first functional application prototype with document upload, authentication UI, architecture planning, and initial document analysis capabilities.**	Implement Document Analysis and Structured Data Output (JSON)	8		8	
		Deliverable: Software Architecture Design	8		8	
		Frontend: Github Actions CI/CD Pipeline	3		3	
		Interface includes a document upload screen	3		2	
		App Login Screen	2		2	
3	**Build the core document management UI, define the app navigation structure, and implement document capture using the camera while establishing the backend cloud infrastructure.**	Frontend: Documents Overview	3		5	
		Capturing Documents using Camera/Scanning	5		3	
		App Navigation Tree	5		5	
		Setup GCP	5		5	
4						
5						
6						

Sprint	Goal	Feature Name	Est. size	Est. remaining	Real size	Real remaining
		PLEASE CREATE THE BURNDOWN CHART ON A NEW TAB USING THE DATA FROM THIS TAB				

Sprint	Goal	Feature Name	Est. size	Est. remaining	Real size	Real remaining
Release						
Total			0	0		
Sprints						
1			0	0	0	0
2			0	0	0	0
3			0	0	0	0
...				0		0
Features						
1						
2						
3						
		PLEASE CREATE THE BURNDOWN CHART ON A NEW TAB USING THE DATA FROM THIS TAB				

#	Feature Definition of Done	Sprint Release Definition of Done	Project Release Definition of Done
	The feature code has been reviewed and approved by at least one team member (e.g. pull request approved).	The project builds, deploys, and runs successfully in a clean environment.	Application passes (user) testing across Android and iOS mobile devices.
	The feature code is merged into the main branch without breaking other functionality.	Sprint release notes are written, summarising the modifications made during the sprint.	User documentation is updated to provide instructions on how to build and use the application.
	The feature was manually tested and works as expected without critical bugs.		Design documentation is updated to reflect the architecture and system design.
	Feature components are demo-ready.		
	The feature is documented with clear explanations of its functionality and usage.		

Type	Link / reference

#	Context	Name	Version	License	Comment
	frontend	@react-navigation/bottom-tabs	^7.15.5	MIT	also counts for all expo subpackages
	frontend	@react-navigation/elements	^2.9.10	MIT	
	frontend	@react-navigation/native	^7.1.33	MIT	
	frontend	@tamagui/config	^2.0.0-rc.41	MIT	
	frontend	@tamagui/lucide-icons-2	^2.0.0-rc.41	MIT	
	frontend	expo	~55.0.15	MIT	
	frontend	expo-constants	~55.0.14	MIT	
	frontend	expo-device	~55.0.15	MIT	
	frontend	expo-document-picker	~55.0.13	MIT	
	frontend	expo-font	~55.0.6	MIT	
	frontend	expo-glass-effect	~55.0.10	MIT	
	frontend	expo-image	~55.0.8	MIT	
	frontend	expo-image-picker	~55.0.20	MIT	
	frontend	expo-linking	~55.0.13	MIT	
	frontend	expo-router	~55.0.12	MIT	
	frontend	expo-splash-screen	~55.0.18	MIT	
	frontend	expo-status-bar	~55.0.5	MIT	
	frontend	expo-symbols	~55.0.7	MIT	
	frontend	expo-system-ui	~55.0.15	MIT	
	frontend	expo-web-browser	~55.0.14	MIT	
	frontend	react	19.2.0	MIT	
	frontend	react-dom	19.2.0	MIT	
	frontend	react-hook-form	^7.73.1	MIT	
	frontend	react-native	0.83.4	MIT	
	frontend	react-native-gesture-handler	~2.30.0	MIT	
	frontend	react-native-reanimated	4.2.1	MIT	
	frontend	react-native-safe-area-context	~5.6.2	MIT	
	frontend	react-native-screens	~4.23.0	MIT	
	frontend	react-native-svg	15.15.3	MIT	
	frontend	react-native-web	~0.21.0	MIT	
	frontend	react-native-worklets	0.7.2	MIT	
	frontend	tamagui	^2.0.0-rc.41	MIT	
	frontend	@types/react	~19.2.2	MIT	
	frontend	eslint	^9.0.0	MIT	
	frontend	eslint-config-expo	~55.0.0	MIT	
	frontend	eslint-config-prettier	^10.1.8	MIT	
	frontend	eslint-config-universe	^15.0.3	MIT	
	frontend	eslint-plugin-prettier	^5.5.5	MIT	
	frontend	prettier	^3.8.3	MIT	
	frontend	typescript	~5.9.2	Apache-2.0	
	frontend	jotai	^5.100.9	MIT	

#	Context	Name	Version	License	Comment
	frontend	@tanstack/query	^2.20.0	MIT	
	backend	neo4j	6.1.0	Apache 2.0 AND Python-2.0	
	backend	numpy	2.2.6	BSD License	
	backend	openai	2.34.0	Apache Software License	
	backend	opencv-python	4.12.0.88	Apache Software License	
	backend	paddleocr	3.5.0	Apache License 2.0	
	backend	pandas	3.0.2	BSD License	
	backend	pillow	12.2.0	MIT-CMU	
	backend	pydantic	2.13.3	MIT	
	backend	python-dotenv	1.2.2	BSD-3-Clause	
	backend	requests	2.33.1	Apache Software License	
	backend	protobuf	3.20.2	BSD-3-Clause	
	backend	fast-api	0.136.1	MIT	
	backend	uvicorn	0.46.0	BSD-3-Clause	
	backend	firebase-admin	7.4.0	Apache-2.0	
	backend	google-cloud-firestore	2.27.0	Apache-2.0	
	backend	python-multipart	0.0.27	Apache-2.0	

Last Name	First Name	Value					
Brüggemann	**Jonas**	0		**0.00**	**OK**		
Khan	**Muhamad**	0					
Ebied	**Andrew**						
Vinzens	**Hannes**						
Ahmed	**Hasnat**			0	No size		
Amer	**Hossam**			1	Trivial size		
Sandt	**Eloi**			2	Small size		
Busse	**Tobias**			3	Medium size		
Zeeshan	**Muhamad**			5	Large size		
Haseeb	**Abdul**			8	Very large size		
				13	Too large (size)		
How to play planning poker							
1. Everyone type their number into their value field, don't hit return yet							
2. Someone, perhaps a product owner, count down 3.. 2.. 1..							
3. Then, everyone hit return to submit their value							